

```
}//~!
```

You can initialise a multi-dimensional array with an appropriate set of values within braces. For example:

```
int a[3][2] = {{1,2}, {2,6}, {3,7}};
```

4.2.4 Pointers

Pointers are special objects for holding (or “pointing to”) memory addresses. In C, address of an object in memory can be obtained by means of unary operator `&`. To reach the pointed object, we use indirection operator `(*)` on a pointer.

A pointer of type “pointer to object of type” holds the address of (that is, points to) an object of type. Since pointers are objects, you can have a pointer pointing to a pointer (and so on). Other objects commonly pointed at include arrays, structures, and unions.

A pointer to a function is best thought of as an address, usually in a code segment, where that function’s executable code is stored; that is, the address to which control is transferred when that function is called.

Although pointers contain numbers with most of the characteristics of unsigned integers, they have their own rules and restrictions for declarations, assignments, conversions, and arithmetic. The examples in the next few sections illustrate these rules and restrictions.

Pointer declarations

Pointers are declared same as any other variable, but with `*` ahead of identifier. Type at the beginning of declaration specifies the type of a pointed object. A pointer must be declared as pointing to some particular type, even if that type is void, which really means a pointer to anything. Pointers to void are often called generic pointers, and are treated as pointers to char in CCS.

If type is any predefined or user-defined type, including void, the declaration `type *p; /* Uninitialized pointer */` declares `p` to be of type “pointer to type”. All the scoping, duration, and visibility rules apply to the `p` object just declared. You can view the declaration in this way: if `*p` is an object of type, then `p` has to be a pointer to such objects.

Note: You must initialise pointers before using them! Our previously declared pointer `*p` is not initialized (i.e. assigned a value), so it cannot be used yet. In case of multiple pointer declarations, each identifier requires an indirect operator.

For example: